

Spring Framework - II



Agenda

1

Spring MVC -Web

2

Spring Database Interaction



Module 1: Spring - MVC



Objectives

- This module is aimed at :
 - Core Components of Spring MVC
 - Flow of Request in Spring MVC based Web Application
 - How to develop a Web Application using Spring MVC

Spring MVC

A Spring MVC is a Java framework which is used to build web applications.

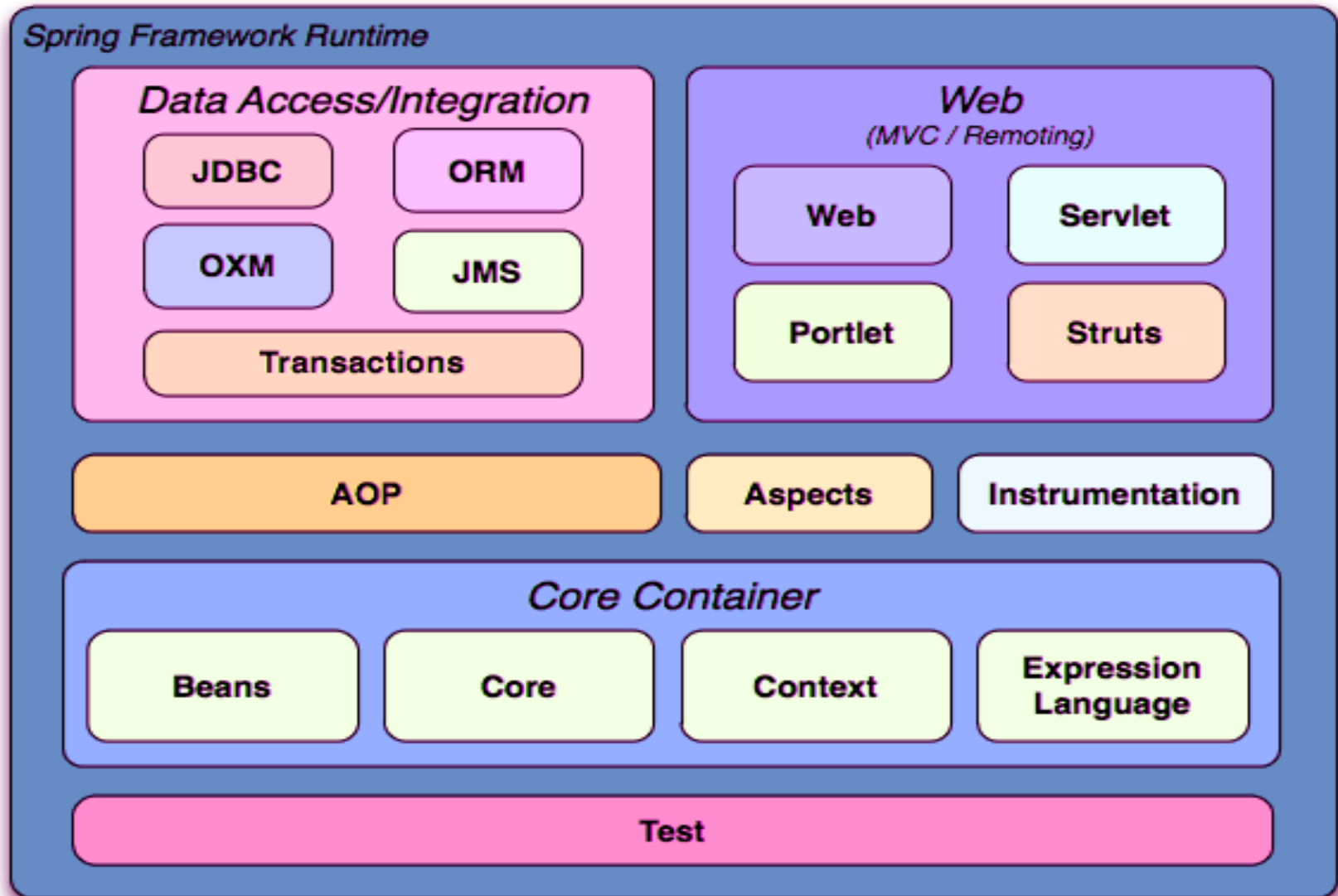
It follows the Model-View-Controller design pattern.

It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection.

A Spring MVC provides an elegant solution to use MVC in spring framework by the help of DispatcherServlet.

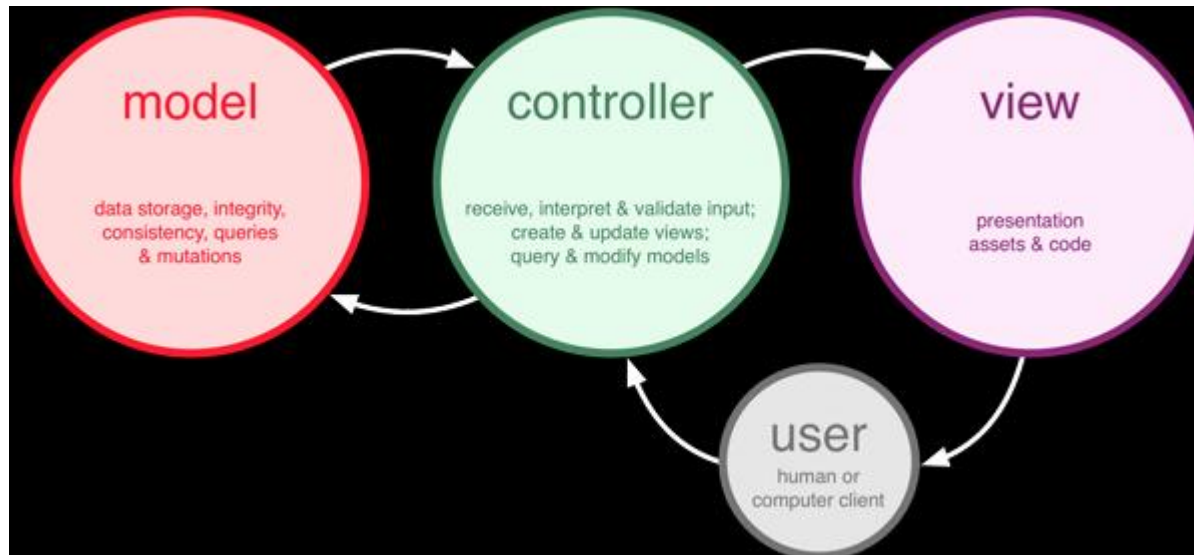
DispatcherServlet is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.

Spring Modules



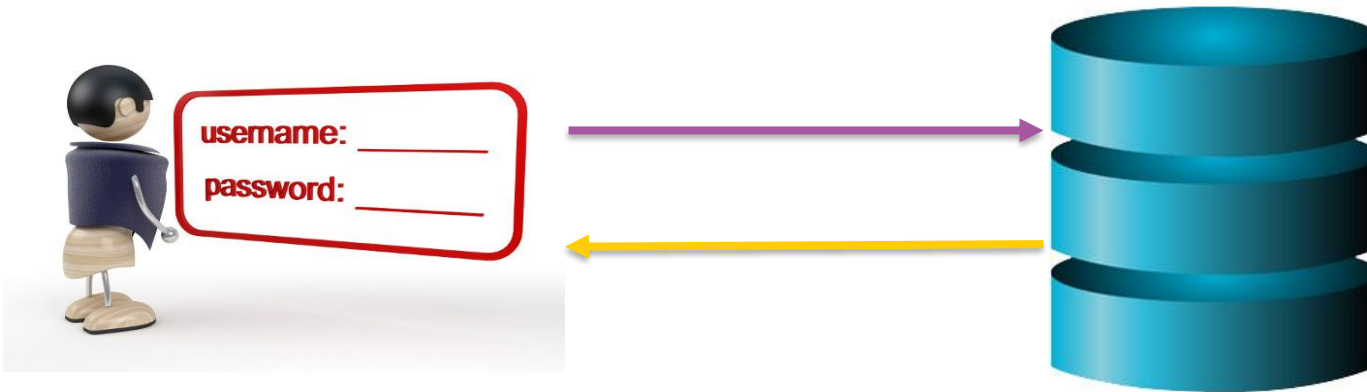
What is MVC?

- The **Model-View-Controller (MVC)** is an **architectural pattern** that separates an application into three main logical components: the **model**, the view, and the controller.
- MVC is a software design pattern for developing Web Applications.
- Each of these components are built to handle specific development aspects of an application.
- MVC is one of the most frequently used industry-standard web development framework to create scalable and extensible projects.



Model

- The Model component corresponds to all the data-related logic that the user works with.
- This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.
- For example, a Customer object will retrieve the customer information from the database, manipulate it and update it data back to the database or use it to render data.



View

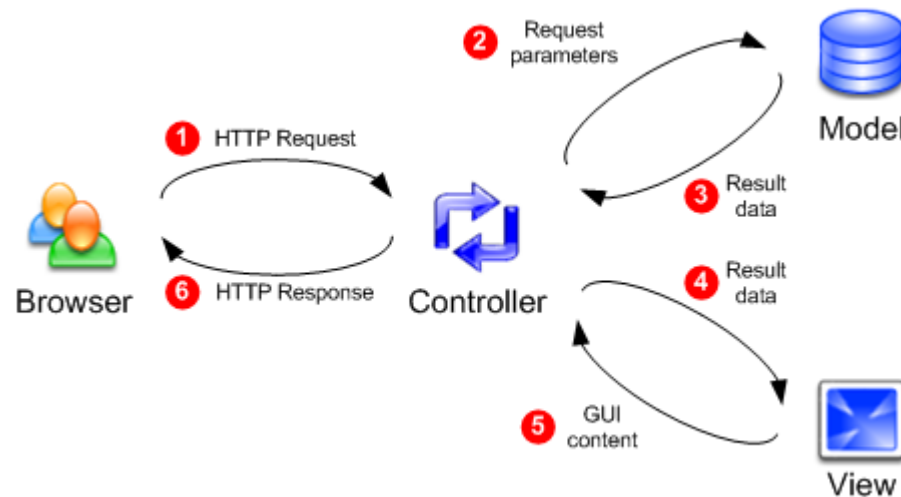
- The View component is used for all the UI logic of the application.
- For example, the Customer view will include all the UI components such as text boxes, dropdowns, etc. that the final user interacts with.

The screenshot displays a web application interface. At the top, a blue header bar contains the title 'My MVC Application' on the left and a '[Log On]' link on the right. Below the header, there are two buttons: 'Home' and 'About'. The main content area is titled 'Index' and includes a 'Create New' link. Below this, there is a table with three columns: 'Name', 'City', and 'Founded'. The table contains two rows of data, each representing a football club. Each row has three links: 'Edit', 'Details', and 'Delete'.

	Name	City	Founded
Edit Details Delete	Manchester United	Manchester, UK	01/04/1878 00:00
Edit Details Delete	Sheffield Wednesday	Sheffield, UK	04/09/1867 00:00

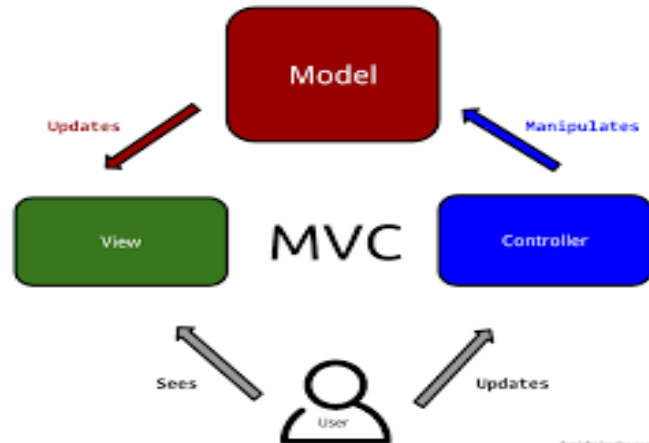
Controller

- Controllers act as an interface between Model and View components.
- It processes all the business logic and incoming requests, manipulate data using the Model component and interact with the Views to render the final output.
- For example, the Customer controller will handle all the interactions and inputs from the Customer View and update the database using the Customer Model. The same controller will be used to view the Customer data.



Spring – MVC Framework

- The Spring Web MVC framework provides Model-View-Controller (MVC) architecture.
- It creates ready components that can be used to develop flexible and loosely coupled web applications.
- The MVC pattern results in separating the different aspects of the application :
 - (input logic, business logic, and UI logic),
- While providing a loose coupling between these elements.



MVC Overview

- MVC => Model-View-Controller
- Pattern helps to clearly separates business, navigation and presentation logic
- Proven mechanism for building a thin & clean web-tier
- Three core collaborating components
 - **Controller**
 - Handles navigation logic and interacts with the service tier for business logic
 - **Model**
 - Handles the business logic and data
 - **View**
 - Renders the response to the request
 - Pulls data from the model

MVC Components

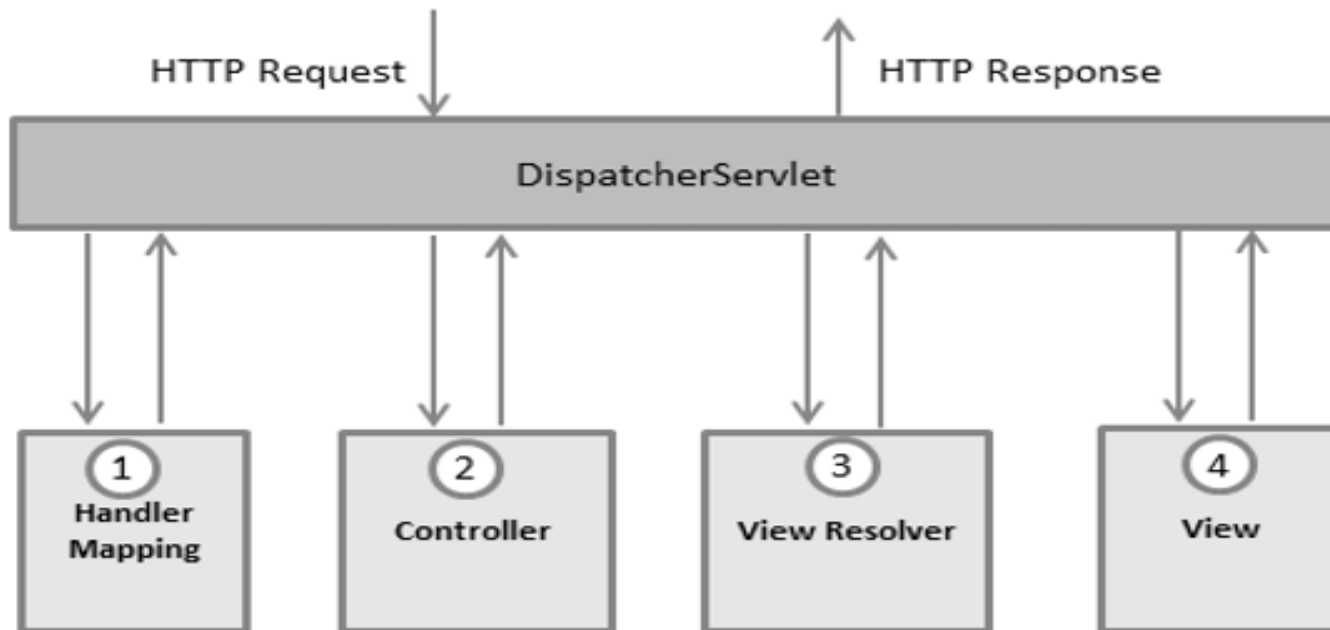
- The **Model** encapsulates the application data and in general, they will consist of POJO.
- A model contains the data of the application. A data can be a single object or a collection of objects.
- The **View** is responsible for rendering the model data and in general, it generates HTML output that the client's browser can interpret.
- A view represents the provided information in a particular format. Generally, JSP+JSTL is used to create a view page.
- Although spring also supports other view technologies such as Apache Velocity, Thymeleaf and FreeMarker.
- The **Controller** is responsible for processing User Requests and Building Appropriate Model and passes it to the view for rendering.
- A controller contains the business logic of an application. Here, the **@Controller** annotation is used to mark the class as the controller.

MVC in Spring

- A single **Front Controller** servlet that dispatches requests to individual Controllers
 - Proven pattern shown in Struts and Core J2EE Patterns
- Front Controller - In Spring Web MVC, the DispatcherServlet class works as the front controller. It is responsible to manage the flow of the Spring MVC application.
- Request routing is completely controlled by the Front Controller
 - Individual Controllers can be used to handle many different URLs
- **Controllers** are Classes of Java
- Controllers are managed exactly like any other bean in the Spring ApplicationContext

The DispatcherServlet

- The Spring Web model-view-controller (MVC) framework is designed around a DispatcherServlet that handles all the HTTP requests and responses.
- The request processing workflow of the Spring Web MVC DispatcherServlet is illustrated in the following diagram –



HTTP DispatcherServlet Events

- After receiving an HTTP request, DispatcherServlet consults the **HandlerMapping** to call the appropriate Controller.
- The **Controller** takes the request and calls the appropriate **service** methods based on used GET or POST method. The service method will set model data based on defined business logic and returns view name to the DispatcherServlet.
- The DispatcherServlet will take help from **ViewResolver** to pickup the defined view for the request.
- Once view is finalized, The DispatcherServlet passes the **model** data to the view which is finally rendered on the browser.
- **All above components are parts of *WebApplicationContext* which is an extension of the *plainApplicationContext* with some extra features necessary for web applications.**

Core Components of Spring MVC

- **DispatcherServlet**

- Spring's Front Controller implementation

- **Controller**

- User created component for handling requests
- Encapsulates navigation logic
- Delegates to the service objects for business logic
- The default handler is a very simple **Controller** interface with method:
 ModelAndView **handleRequest**(request, response).

- **View**

- Responsible for rendering output

Core Components of Spring MVC (Contd.).

- **ModelAndView**

- Created by the Controller
- Stores the Model data
- Associates a View to the request (usually a logical view names)
- The ModelAndView class is simply a container by which the model may be transported to and exposed by the view.

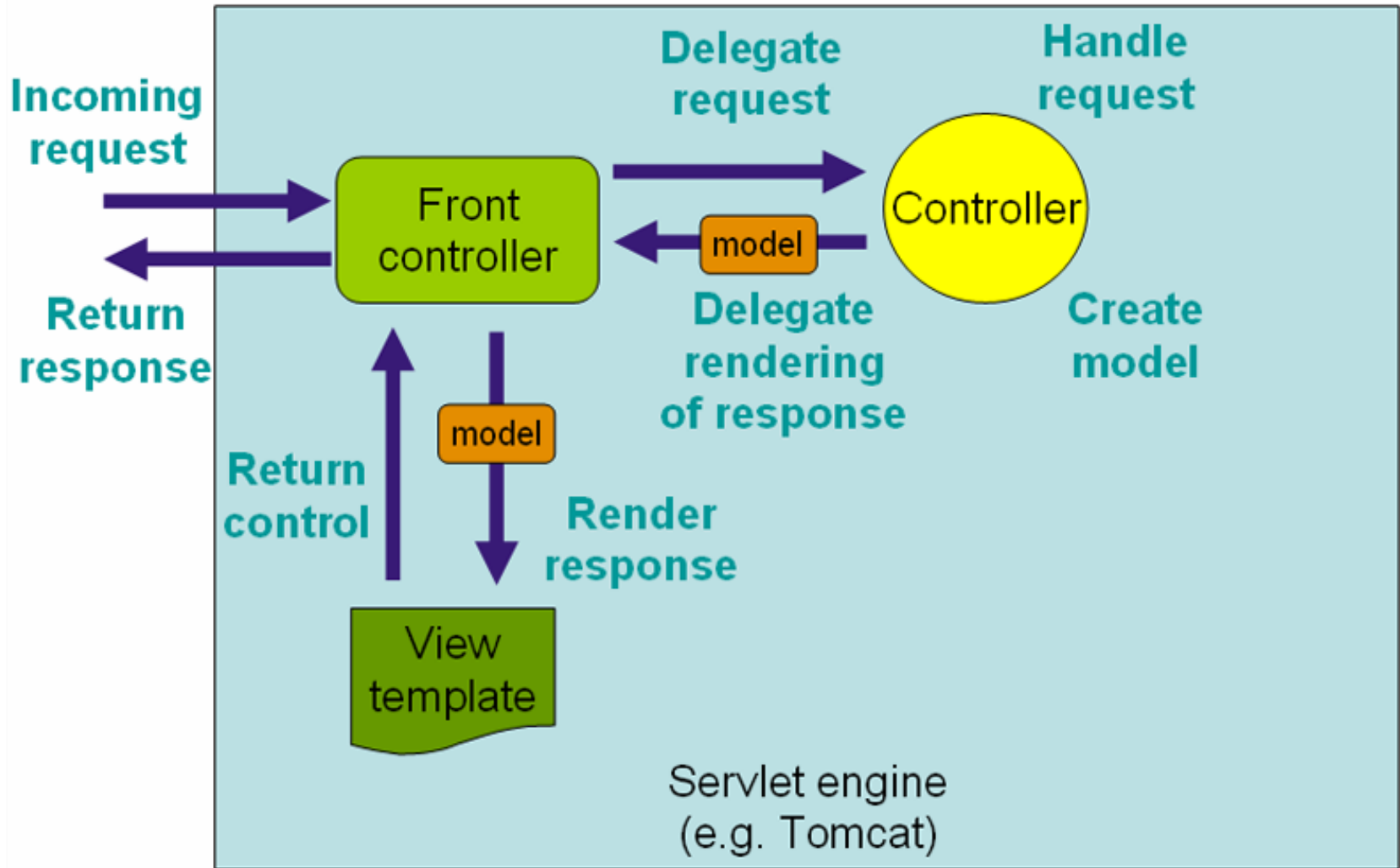
- **ViewResolver**

- Used to map logical View names to actual View implementations

- **HandlerMapping**

- Strategy interface used by DispatcherServlet
- for mapping incoming requests to individual Controllers

Work Flow in Spring MVC (High Level)



The Deployment Descriptor: web.xml

- A web application's deployment descriptor describes the classes, resources and configuration of the application and how the web server uses them to serve web requests.
- When the web server receives a request for the application, it uses the deployment descriptor to map the URL of the request to the code that ought to handle the request.
- Java web applications use a deployment descriptor file to determine how URLs map to servlets, which URLs require authentication, and other information.
- This file is named web.xml, and resides in the app's WAR under the WEB-INF/ directory.
- web.xml is part of the servlet standard for web applications.

Web.xml

- The deployment descriptor is a file named web.xml.
- It resides in the app's WAR under the WEB-INF/ directory.
- The file is an XML file whose root element is **<web-app>**.
- Here is a simple web.xml example that maps all URL paths (/*) to the servlet class .

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="2.5">
  <servlet>
    <servlet-name>comingsoon</servlet-name>
    <servlet-class>mysite.server.ComingSoonServlet</servlet-
class>
  </servlet>
  <servlet-mapping>
    <servlet-name>comingsoon</servlet-name>
    <url-pattern>/*</url-pattern>
  </servlet-mapping>
</web-app>
```

Servlets and URL paths

- web.xml defines mappings between URL paths and the servlets that handle requests with those paths.
- The web server uses this configuration to identify the servlet to handle a given request and call the class method that corresponds to the request method.
- For example: the doGet() method for HTTP GET requests.
- To map a URL to a servlet, you declare the servlet with the <servlet> element, then define a mapping from a URL path to a servlet declaration with the <servlet-mapping> element.

Servlet Mapping

- The `<servlet-mapping>` element specifies a URL pattern and the name of a declared servlet to use for requests whose URL matches the pattern.
- The URL pattern can use an asterisk (*) at the beginning or end of the pattern to indicate zero or more of any character.
- The standard does not support wildcards in the middle of a string, and does not allow multiple wildcards in one pattern.
- The pattern matches the full path of the URL, starting with and including the forward slash (/) following the domain name.

```
<servlet-mapping>
  <servlet-name>redteam</servlet-name>
  <url-pattern>/red/*</url-pattern>
</servlet-mapping>

<servlet-mapping>
  <servlet-name>blueteam</servlet-name>
  <url-pattern>/blue/*</url-pattern>
</servlet-mapping>
```

JSP in Web.xml

- An app can use JavaServer Pages (JSPs) to implement web pages. JSPs are servlets defined using static content, such as HTML, mixed with Java code.
- For example, if an app has a JSP file named start.jsp in a subdirectory named register/ in its WAR, App Engine compiles it and maps it to the URL path /register/start.jsp.

```
<servlet>
    <servlet-name>register</servlet-name>
    <jsp-file>/register/start.jsp</jsp-file>
</servlet>

<servlet-mapping>
    <servlet-name>register</servlet-name>
    <url-pattern>/register/*</url-pattern>
</servlet-mapping>
```

The welcome file list

- The welcome-file-list element of web-app, is used to define a list of welcome files. Its sub element is welcome-file that is used to define the welcome file.
- A welcome file is the file that is invoked automatically by the server, if you don't specify any file name.
- By default server looks for the welcome file in following order:
welcome-file-list in web.xml
 - index.html
 - index.htm
 - index.jsp
 - If none of these files are found, server renders 404 error.

The welcome file list

```
<welcome-file-list>  
    <welcome-file>index.jsp</welcome-file>  
    <welcome-file>index.html</welcome-file>  
</welcome-file-list>
```

- Now, index.jsp and index.html will be the welcome files.
- If you have the welcome file, you can directly invoke the project as given below:

<http://localhost:8888/myproject>

Configuring Spring-MVC Application

- Configure **DispatcherServlet** in **web.xml** and establish URL mappings for it.

E.g.:

```
<servlet>
```

```
<servlet-name>springweb</servlet-name>
```

```
<servlet-
```

```
  class>org.springframework.web.servlet.DispatcherServlet
```

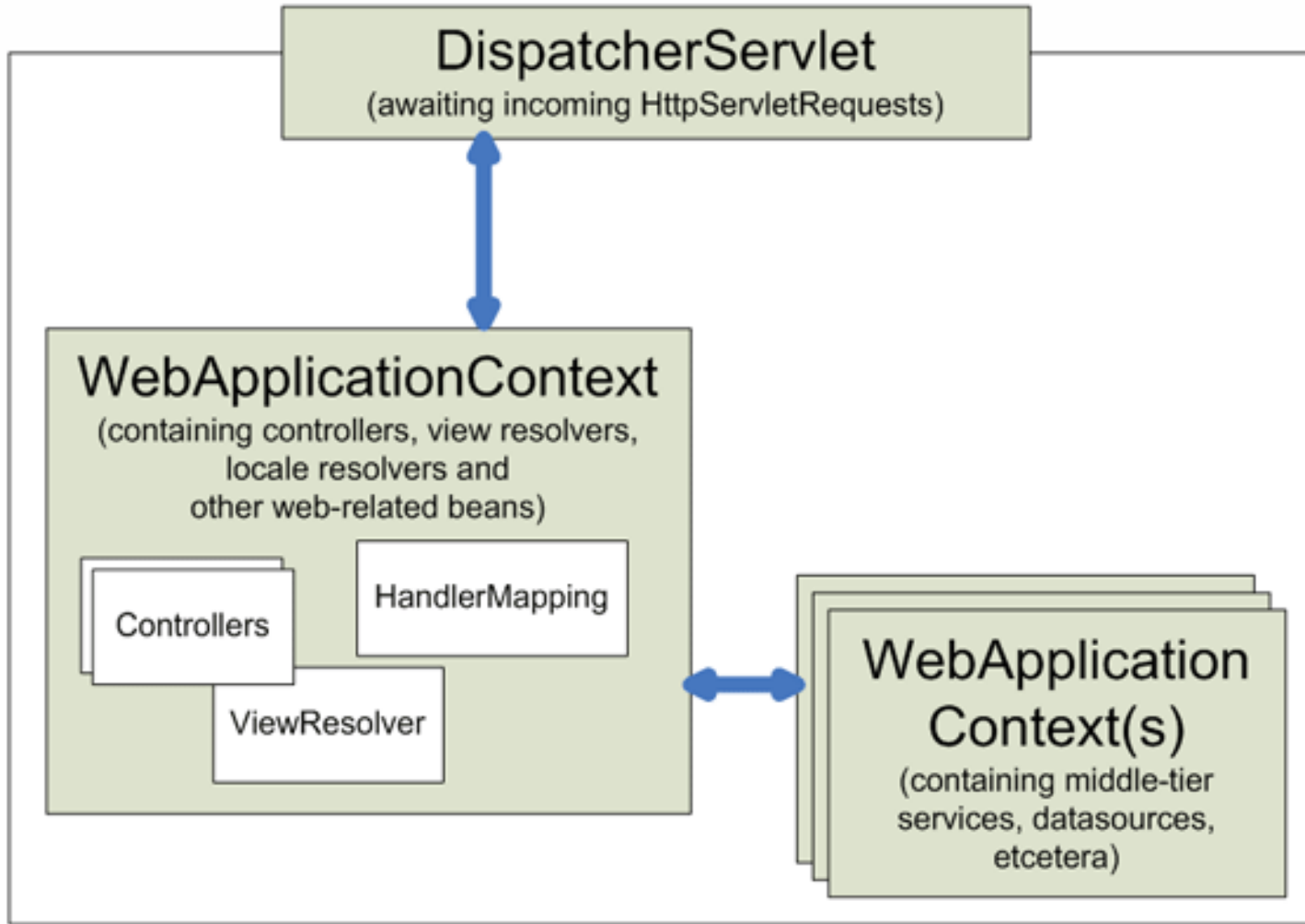
```
</servlet-class> ...
```

- Spring configuration metadata in a configuration file named [servlet-name]-servlet.xml in the WEB-INF directory of your web application
 - E.g.: the DispatcherServlet defined in web.xml with name as **springweb**, will look for the configuration file **/WEB-INF/springweb-servlet.xml**

Configuring Spring-MVC Application (Contd.).

- When a request is submitted to the server, DispatcherServlet looks in springweb-servlet.xml for a mapping of the specified URL to some controller bean.
- This is done by using a **BeanNameUrlHandlerMapping class** by default.
 - There are other mapping handlers that you can specify instead of the default. (**SimpleUrlHandlerMapping class**)
- The default handler looks for a bean that has the URL as its name and gives it to DispatcherServlet.
- DispatcherServlet looks for a **view resolver** to resolve the view name that it got from the ModelAndView object - (**InternalResourceViewResolver class**)

Context hierarchy in Spring Web MVC



Controller(s) in Spring-MVC

- Spring's Web MVC framework is designed around a DispatcherServlet that dispatches requests to handlers (Controllers)
- The default handler is a very simple **Controller** interface, just offering the below method :
`ModelAndView handleRequest(request,response)`
- Application Controllers are typically sub-classes of the included implementation hierarchy, consisting of:
 - **AbstractController**
 - **AbstractCommandController**
 - **SimpleFormController**

Mapping Handler

```
<mvc:annotation-driven />
```

```
<context:component-scan base-package="com.rps.controller" />
```

```
<mvc:default-servlet-handler />
```

```
<bean id="viewResolver"
```

```
class="org.springframework.web.servlet.view.UrlBasedViewResolver">
```

```
<property name="viewClass"
```

```
value="org.springframework.web.servlet.view.JstlView" />
```

```
<property name="prefix" value="/WEB-INF/views/" />
```

```
<property name="suffix" value=".jsp" />
```

```
</bean>
```

Mapping Handler

This is the important configuration file where we need to specify the ViewResolver and View components.

This xml file should be located inside the WEB-INF directory.

`<mvc:annotation-driven />` declares explicit support for annotation-driven MVC controllers

- (i.e. `@RequestMapping`, `@Controller`, although support for those is the default behaviour),

As well as adding support for declarative validation via `@Valid` and message body marshalling with `@RequestBody/ResponseBody`.

The `<context:component-scan>` element defines the base-package where `DispatcherServlet` will search the controller class.

Mapping Handler-View Resolver

The InternalResourceViewResolver class is used for the ViewResolver.

The prefix+string returned by controller+suffix page will be invoked for the view component.

```
<context:component-scan base-package="com.rps" />
<bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/jsp/" />
<property name="suffix" value=".jsp" /> </bean>
```

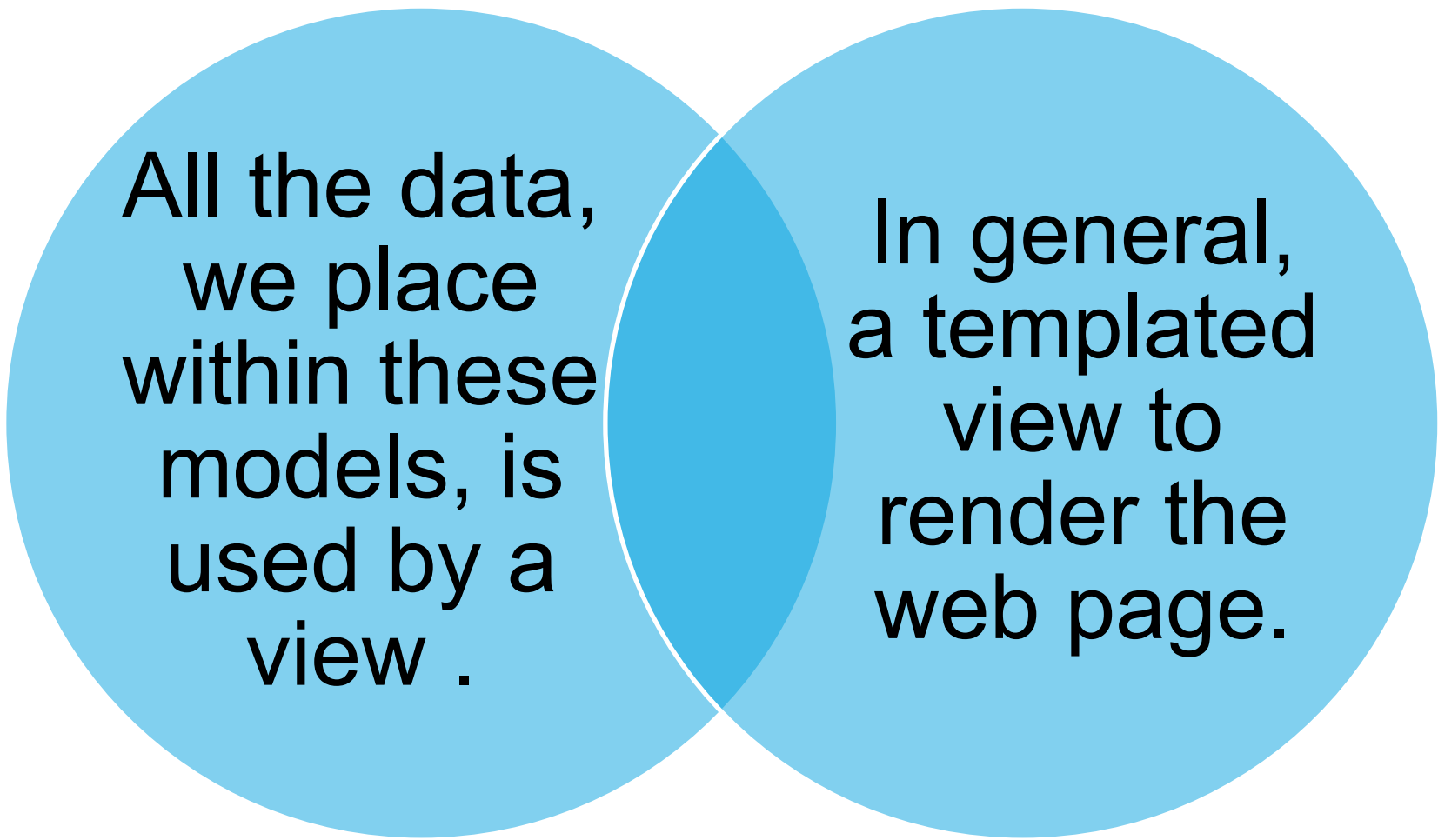
Spring MVC - Page Redirection

- In Spring MVC there is a class which is used to transfer an http request to another page.
`org.springframework.web.servlet.view.RedirectView`,
- As name indicated, a view is redirect to another absolute, context relative, or current request relative URL.
- There is attribute “redirect” which can be prefixed to the view name to indicate that the new request should be a redirected one.

Summary

- In this module, we have learnt :
 - Core Components of Spring MVC
 - Flow of Request in Spring MVC based Web Application
 - How to develop a Web Application using Spring MVC

The View



All the data,
we place
within these
models, is
used by a
view .

In general,
a templated
view to
render the
web page.

MVC Form Tag Library

The Spring MVC form tags are the configurable and reusable building blocks for a web page.

These tags provide JSP, an easy way to develop, read and maintain.

The Spring MVC form tags can be seen as data binding-aware tags that can automatically set data to Java object/bean and also retrieve from it.

Here, each tag provides support for the set of attributes of its corresponding HTML tag counterpart, making the tags familiar and easy to use.

Configuration of Spring MVC Form Tag

The form tag library comes under the **spring-webmvc.jar**.

To enable the support for form tag library, it is required to reference some configuration.

So, add the following directive at the beginning of the JSP page:

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form" %>
```

Spring MVC Form Tags

Form Tag	Description
form:form	It is a container tag that contains all other form tags.
form:input	This tag is used to generate the text field.
form:radiobutton	This tag is used to generate the radio buttons.
form:checkbox	This tag is used to generate the checkboxes.
form:password	This tag is used to generate the password input field.
form:select	This tag is used to generate the drop-down list.
form:textarea	This tag is used to generate the multi-line text field.
form:hidden	This tag is used to generate the hidden input field.

Spring's Form Tag Library

The form tag

The input tag

The checkbox tag

The checkboxes tag

The radiobutton tag

The radiobuttons tag

The password tag

The option tag

The select tag

The options tag

The textarea tag

The hidden tag

The errors tag

HTML5 tags

The form tag

The Spring MVC form tag is a container tag.

It is a parent tag that contains all the other tags of the tag library.

This tag generates an HTML form tag and exposes a binding path to the inner tags for binding.

```
<form:form action="mapPath" modelAttribute="cust">
```

- **modelAttribute** attribute is the key which specifies a name of the model object that backs
- This will correspond to the **@ModelAttribute** later on in the controller.

Spring MVC Form Text Field

- The Spring MVC form text field tag generates an HTML input tag using the bound value.
- By default, the type of the input tag is text.

`<form:input path="name" />`

- Here, path attribute binds the form field to the bean property.
- The Spring MVC form tag library also provides other input types such as email, date, tel, etc.

path attribute

- Each input fields specifies a path attribute.
- This must correspond to a getter/setter of the model attribute (Eg: the Employee class).
- When the page is loaded, the input fields are populated by Spring, which calls the getter of each field bound to an input field.
- When the form is submitted, the setters are called to save the values of the form to the object.
- When **the form is submitted**, the POST handler in the controller is invoked and the form is automatically bound to the *employee* argument that we passed in

`<form:input path="name" />`

Spring MVC Form Radio Button, CheckBox

- The Spring MVC form radio button allows us to choose only one option at a time.
- This tag renders an HTML input tag of type radio.

```
<form:radio button path="abc" value="xyz"/>
```

```
<form:checkbox path="abc" value="element"/>
```

- The Spring MVC form checkbox facilitates to choose multiple options at the same time.

Spring MVC Form Drop-Down List

- The Spring MVC form drop-down list contains the list of elements.
- This tag generates an HTML select element.
- It allows data binding to the selected element.

<form:select path="city">

option tag

- This tag generates an HTML option tag.
- Each tag contains a value that can be selected by the user.

<form:option value="Bangalore" label="Bangalore"/>

@ModelAttribute Annotation

- @ModelAttribute annotation is used to bind a method parameter or a method return value to a named model attribute.
- It can be used either on methods or on method parameters in controller.
- Spring invokes all methods that have @ModelAttribute annotation before handler methods (i.e. methods annotated with the @RequestMapping) in a same controller.

@ModelAttribute annotation at Parameter level

- When using the @ModelAttribute annotation as a method parameter, it binds the form data with a POJO bean.
- It has a value attribute which acts as a name of the model attribute to bind.

```
@RequestMapping(value="/handleRequest", method=RequestMethod.GET)
```

```
public String saveCustomer(@ModelAttribute("cust") Customer custObj)
{
    custObj.setName("John");
    return "listView"
}
```

- In this case, a new instance of the Customer is created and then passed to the handler method for further processing.

@ModelAttribute annotation at Method level

- When using the @ModelAttribute annotation at the method level, developers can add the values in the Model at a global level.
- It means for every request, a default value will be there in the controller for every response.

@ModelAttribute

```
public void addAttributes(Model model) {  
    model.addAttribute("welcome_text", "Welcome to my Web Site");  
}
```

Model Interface

- In Spring MVC, the model works as a container that contains the **data** of the application.
- Here, a data can be in any form such as objects, strings, information from the database, etc.
- The model can supply attributes used for rendering views.
- To provide a view with usable data, we simply add this data to its *Model* object.
- Additionally, maps with attributes can be merged with *Model* instances.
- It contains four **addAttribute()** and one **mergeAttribute()** method.

ModelMap Interface

- Just like the *Model* interface above, *ModelMap* is also used to pass values to render a view.
- The advantage of *ModelMap* is it gives us the ability to pass a collection of values and treat these values as if they were within a *Map*

```
@GetMapping("/printViewPage")  
public String passParametersWithModelMap(ModelMap map) {  
  
    map.addAttribute("welcomeMessage", "welcome");  
    map.addAttribute("message", "Spring MVC");  
  
    return "viewPage";  
}
```

JSTL (JSP Standard Tag Library)

- The JSP Standard Tag Library (JSTL) represents a set of tags to simplify the JSP development.

Advantages:

- **Fast Development** JSTL provides many tags that simplify the JSP.
- **Code Reusability** We can use the JSTL tags on various pages.
- **No need to use scriptlet tag** It avoids the use of scriptlet tag.

JSTL Tags

Tag Name	Description
Core tags	• The JSTL core tag provide variable support, URL management, flow control, etc. • The URL for the core tag is http://java.sun.com/jsp/jstl/core. • The prefix of core tag is c.
Function tags	The functions tags provide support for string manipulation and string length. The URL for the functions tags is http://java.sun.com/jsp/jstl/functions and prefix is fn.
Formatting tags	The Formatting tags provide support for message formatting, number and date formatting, etc. The URL for the Formatting tags is http://java.sun.com/jsp/jstl/fmt and prefix is fmt.
XML tags	The XML tags provide flow control, transformation, etc. The URL for the XML tags is http://java.sun.com/jsp/jstl/xml and prefix is x.
SQL tags	The JSTL SQL tags provide SQL support. The URL for the SQL tags is http://java.sun.com/jsp/jstl/sql and prefix is sql.

JSTL Core Tags

- The JSTL core tag provides variable support, URL management, flow control etc.
- The syntax used for including JSTL core library in your JSP is:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

JSTL Core Tags List

Tags	Description
<u>c:out</u>	It display the result of an expression, similar to the way <code><%=...%></code> tag work.
<u>c:import</u>	It Retrives relative or an absolute URL and display the contents to either a String in 'var',a Reader in 'varReader' or the page.
<u>c:set</u>	It sets the result of an expression under evaluation in a 'scope' variable.
<u>c:remove</u>	It is used for removing the specified scoped variable from a particular scope.
<u>c:catch</u>	It is used for Catches any Throwable exceptions that occurs in the body.
<u>c:if</u>	It is conditional tag used for testing the condition and display the body content only if the expression evaluates is true.
<u>c:choose,</u> <u>c:when,</u> <u>c:otherwise</u>	It is the simple conditional tag that includes its body content if the evaluated condition is true.
<u>c:forEach</u>	It is the basic iteration tag. It repeats the nested body content for fixed number of times or over collection.
<u>c:forTokens</u>	It iterates over tokens which is separated by the supplied delimiters.
<u>c:param</u>	It adds a parameter in a containing 'import' tag's URL.
<u>c:redirect</u>	It redirects the browser to a new URL and supports the context-relative URLs.
<u>c:url</u>	It creates a URL with optional query parameters.

Expression Language (EL) in JSP

The Expression Language (EL) simplifies the accessibility of data stored in the Java Bean component, and other objects like request, session, application etc.`

There are many implicit objects, operators and reserve words in EL.

It is the newly added feature in JSP technology version 2.0.

Syntax for Expression Language (EL): `$\{$  expression  $\}$`

Introduction to EL 4-1

- EL stands for Expression Language

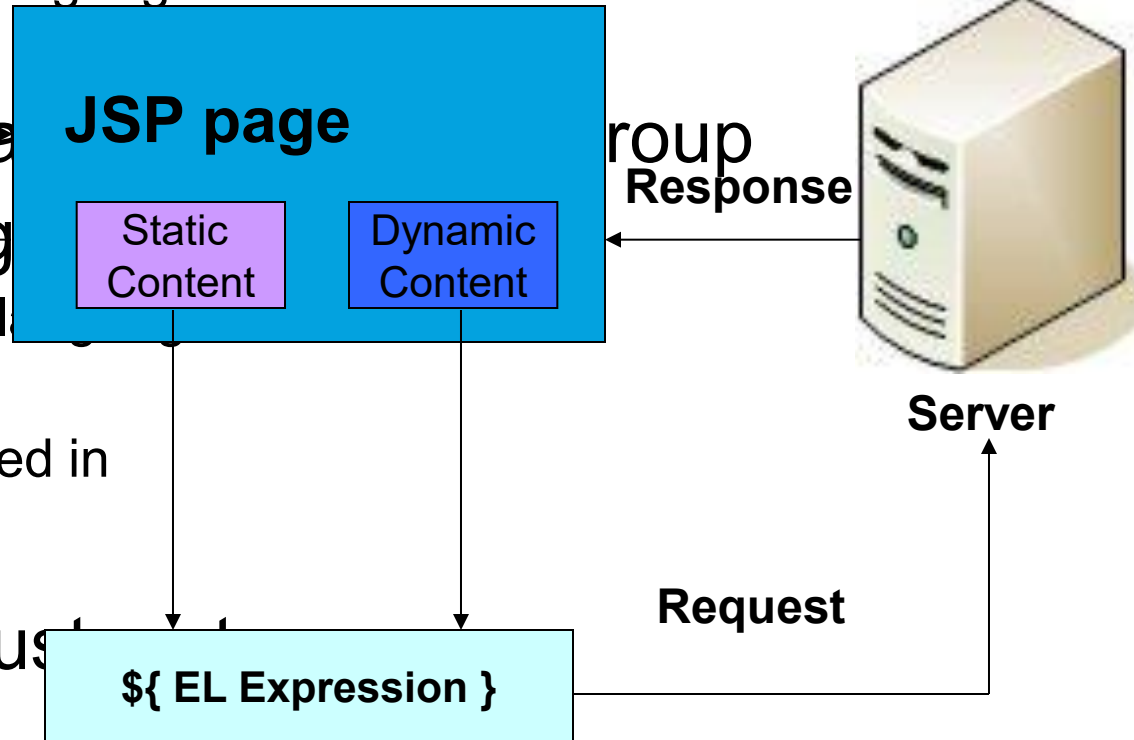
- Developed by two groups

-  Request
Standard Tag
Response
0 expert group
P expression language

-  Client
– `{EL Expression}`
• JSP EL expressions are used in

- Static text

- Standard and Custom



Introduction to EL 4-2

```
<html>
  <head>
    <title>Expression Language</title>
  </head>
  <body>
    <h1>EL operations</h1>
    <table border="1">
      <tr>
        <td><b>Concept</b></td>
        <td><b>EL Expression</b></td>
        <td><b>Result</b></td>
      </tr>
      <tr>
        <td>And</td>
        <td>${'${'}true and true}</td>
        <td>${true and true}</td>
      </tr>
      <tr>
        <td>And</td>
        <td>${'${'}true && false}</td>
        <td>${true && false}</td>
      </tr>
    </table>
  </body>
</html>
```

EL expressions in
<table> tags

EL expressions in
<table> tags

D

Introduction to EL 4-3

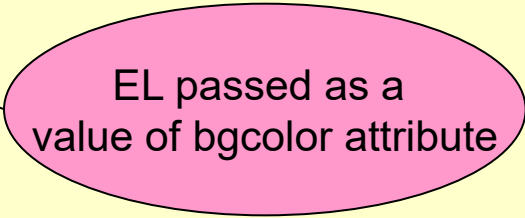
```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-
8">
    <title>Article Expression</title>
  </head>
  <body>
    <h1>Article Expression</h1>
    <h2>Price of the article is ${84.5E4}.</h2>
  </body>
</html>
```



Demonstration: Example 2

Introduction to EL 4-4

```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>EL Example</title>
  </head>
  <%
    pageContext.setAttribute("color", "#FFFFCC");
  %>
  <body bgcolor='${pageScope.color}'>
    <h1>
      Background color changed
    </h1>
  </body>
</html>
```



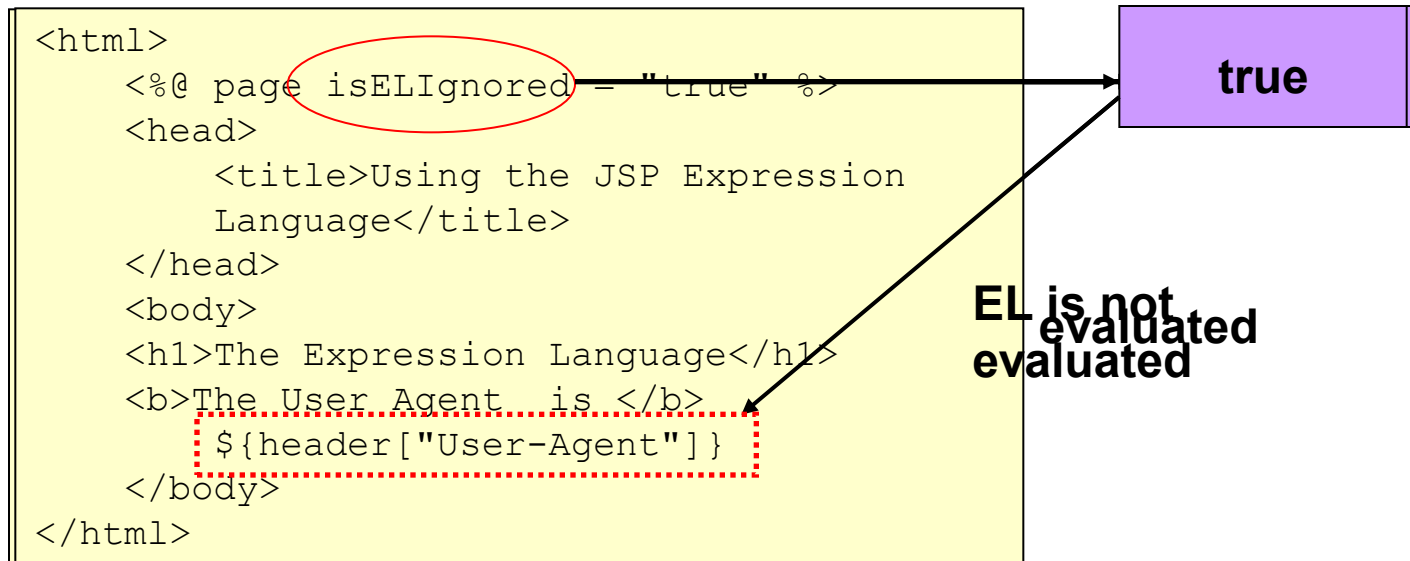
EL passed as a value of bgcolor attribute

The diagram shows a pink oval containing the text "EL passed as a value of bgcolor attribute". An arrow points from this oval to the EL expression `'${pageScope.color}'` within the `bgcolor` attribute of the `<body>` tag in the code block above.

Demonstration: Example 3

Disabling the EL 2-1

- ❑ Disable evaluation to ignore the execution of EL expression in JSP
- ❑ Syntax to disable EL evaluation
`<%@ page isELIgnored ="true|false" %>`



Disabling the EL 2-2

```
<%@page isELIgnored="true" %>
```

isELIgnored
set to true

```
<html>
```

```
<head>
```

```
<title>ELDisabled</title>
```

```
</head>
```

```
<body bgcolor="#ffffff">
```

```
<h1>
```

```
    Enabling/Disabling Expression Language
```

```
</h1>
```

```
<form method="post" action="ELDisabled.jsp">
```

```
<b>  ${'First Name: '}<input type="text"
value="${'Enter your first name'}"/>  </b><br />
```

```
<b>  ${'Last Name: '}<input type="text"
value="${'Enter your last name'}"/>  </b><br />
```

```
<hr/>
```

```
<input type="submit" name="Submit" value="${'Submit'}">
```

```
<input type="reset" value="${'Reset'}">
```

```
</form>
```

```
</body>
```

```
</html>
```

JSP Engine ignores
EL expressions

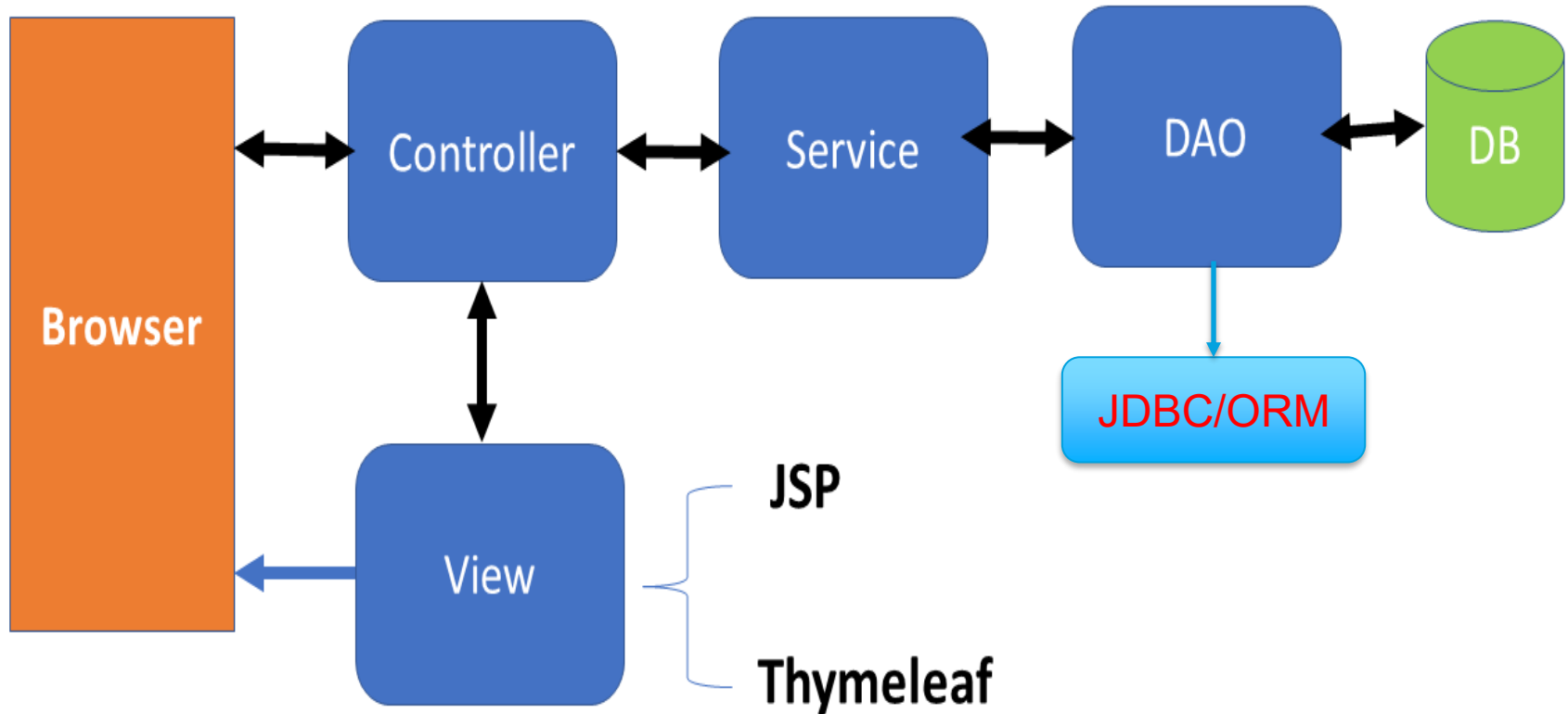
Module 2: Spring Database Interaction



Objectives

- This module is aimed at:
 - Understanding Spring interaction with database using
 - JDBC
 - ORM (Hibernate)

Spring MVC Web Application



Spring JDBCTemplate

- **JdbcTemplate** class is the central class in the JDBC core package.
- It simplifies the use of JDBC in Spring applications by handling the creation and release of resources.
- This class performs basic tasks of core JDBC workflow like SQL statement creation and execution.
- This class executes SQL queries, update statements and stored procedure calls, performs iteration over ResultSets and extraction of returned parameter values and catch and handle JDBC exceptions.

Spring JdbcTemplate

Spring **JdbcTemplate** is a powerful mechanism to connect to the database and execute SQL queries.

It internally uses JDBC api, but eliminates a lot of problems of JDBC API.

JDBC API – Cons

- We need to write a lot of code before and after executing the query, such as:
 - creating connection,
 - statement,
 - closing resultset,
 - connection etc.
- We need to perform exception handling code on the database logic.
- We need to handle transaction.
- Repetition of all these codes from one to another database logic is a time consuming task.

Advantage of Spring JdbcTemplate

Spring JdbcTemplate eliminates all the above mentioned problems of JDBC API.

It provides you methods to write the queries directly, so it saves a lot of work and time.

Spring Jdbc Approaches

Spring framework provides following approaches for JDBC database access:

- JdbcTemplate
- NamedParameterJdbcTemplate
- SimpleJdbcTemplate
- SimpleJdbcInsert and SimpleJdbcCall

JdbcTemplate class

- It is the central class in the Spring JDBC support classes.
- It takes care of creation and release of resources such as creating and closing of connection object etc.
- So it will not lead to any problem if you forget to close the connection.
- It handles the exception and provides the informative exception messages by the help of exception classes defined in the **org.springframework.dao** package.
- We can perform all the database operations by the help of JdbcTemplate class such as insertion, updation, deletion and retrieval of the data from the database.

JdbcTemplate class - Methods

Method	Description
public int update(String query)	is used to insert, update and delete records.
public int update(String query, Object... args)	is used to insert, update and delete records using PreparedStatement using given arguments.
public void execute(String query)	is used to execute DDL query.
public T execute(String sql, PreparedStatementCallback action)	executes the query by using PreparedStatement callback.
public T query(String sql, ResultSetExtractor rse)	is used to fetch records using ResultSetExtractor.
public List query(String sql, RowMapper rse)	is used to fetch records using RowMapper.

DriverManagerDataSource - *.xml

- It is used to contain the information about the database such as driver class name, connection URL, username and password.
- There are a property named **datasource** in the JdbcTemplate class of DriverManagerDataSource type.
- We need to provide the reference of DriverManagerDataSource object in the JdbcTemplate class for the datasource property.
- We are using the JdbcTemplate object in the Dao class, so we are passing it by the setter method but you can use constructor also.

applicationContext.xml

```
<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
<property name="username" value="system" />
<property name="password" value="oracle" />
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="ds"></property>
</bean>

<bean id="edao" class="com.Iti.EmployeeDao">
<property name="jdbcTemplate" ref="jdbcTemplate"></property>
</bean>
```

PreparedStatement in Spring JdbcTemplate

We can execute parameterized query using Spring JdbcTemplate by the help of **execute()** method of JdbcTemplate class.

To use parameterized query, we pass the instance of **PreparedStatementCallback** in the execute method.

```
public T execute(String sql, PreparedStatementCallback<T>);
```

PreparedStatementCallback interface

- It processes the input parameters and output results.
- In such case, you don't need to care about single and double quotes.

Method of PreparedStatementCallback interface

- It has only one method doInPreparedStatement.

```
public T doInPreparedStatement(PreparedStatement  
ps)throws SQLException, DataAccessException
```

RowMapper - Fetching records by Spring JdbcTemplate

RowMapper interface to fetch the records from the database using **query()** method of **JdbcTemplate** class.

RowMapper interface allows to map a row of the relations with the instance of user-defined class.

It iterates the **ResultSet** internally and adds it into the collection.

RowMapper saves a lot of code because it internally adds the data of **ResultSet** into the collection.

It defines only one method **mapRow** that accepts **ResultSet** instance and **int** as the parameter list.

```
public T mapRow(ResultSet rs, int rowNum) throws SQLException
```

Model

- The model can supply attributes used for rendering views.
- To provide a view with usable data, we simply add this data to its Model object.
- Additionally, maps with attributes can be merged with Model instances:

```
public String passParametersWithModel
(Model model) {
    Map<String, String> map = new HashMap<>();
    map.put("spring", "mvc");
    model.addAttribute("message", "SpringMVC");
    model.mergeAttributes(map);
    return "viewPage";
}
```

ModelMap

- Just like the *Model* interface above, *ModelMap* is also used to pass values to render a view.
- The advantage of *ModelMap* is it gives us the ability to pass a collection of values and treat these values as if they were within a *Map*:

```
@GetMapping("/printViewPage")
public String passParametersWithModelMap(ModelMap map) {
    map.addAttribute("welcomeMessage", "welcome");
    map.addAttribute("message", "SpringMVC");
    return "viewPage";
}
```

ModelAndView in Spring MVC

- ModelAndView is an object that holds both the model and view.
- The handler returns the ModelAndView object and DispatcherServlet resolves the view using View Resolvers and View.
- The View is an object which contains view name in the form of the String and model is a map to add multiple objects.
- **The final interface to pass values to a view is the *ModelAndView*.**

```
@GetMapping("/goToViewPage")
```

```
public ModelAndView  
passParametersWithModelAndView() {  
    ModelAndView modelAndView = new  
ModelAndView("viewPage");  
    modelAndView.addObject("message",  
"SpringMVC");  
    return modelAndView;  
}
```

RequestParam Annotation

- In Spring MVC, the `@RequestParam` annotation is used to read the form data and bind it automatically to the parameter present in the provided method.
- So, it ignores the requirement of `HttpServletRequest` object to read the provided data.
- Including form data, it also maps the request parameter to query parameter and parts in multipart requests.
- If the method parameter type is `Map` and a request parameter name is specified, then
 - the request parameter value is converted to a `Map` else
 - the map parameter is populated with all request parameter names and values.

@RequestParam

- The **@RequestParam** is used to read the HTML form data provided by a user and bind it to the request parameter.
- The **Model** contains the request data and provides it to view page.

```
@RequestMapping("/hello")
```

```
    //read the provided form data
```

```
    public String display(@RequestParam("name") String name,@Request  
Param("pass") String pass,Model m)
```

```
    {
```

```
    if(pass.equals("admin"))
```

```
    {
```

```
        String msg="Hello "+ name;
```

```
        //add a message to the model
```

```
        m.addAttribute("message", msg);
```

```
        return "viewpage";  }}
```

Spring with ORM Frameworks

- Spring provides API to easily integrate Spring with ORM frameworks such as :
 - Hibernate,
 - JPA(Java Persistence API),
 - JDO(Java Data Objects),
 - Oracle Toplink and
 - iBATIS.

Advantages of ORM Frameworks with Spring

- **Less coding is required:** By the help of Spring framework, you don't need to write extra codes before and after the actual database logic such as getting the connection, starting transaction, committing transaction, closing connection etc.
- **Easy to test:** Spring's IoC approach makes it easy to test the application.
- **Better exception handling:** Spring framework provides its own API for exception handling with ORM framework.
- **Integrated transaction management:** By the help of Spring framework, we can wrap our mapping code with an explicit template wrapper class or AOP style method interceptor.

Integrating Hibernate with Spring

- The Spring Framework provides integration with **Hibernate**, in terms of:
 - Resource management
 - DAO implementation support
 - transaction strategies
- There are usually two integration styles:
 - using Spring's DAO '**templates**' (helper classes)
 - or coding DAOs against plain Hibernate/JDO/TopLink/etc APIs.
- In both cases, DAOs can be configured through Dependency Injection and participate in Spring's resource and transaction management.

Integrating Hibernate with Spring

- Spring is the leading enterprise application framework.
- Hibernate is the leading ORM framework.
- So combination of these two technologies would be a preferable choice in order to develop robust enterprise applications.
- The DAO implementation uses Hibernate's SessionFactory to query against the database.
- The following configuration approaches can be used :
 - **Spring MVC**: Annotations for controller and XML for bean definitions.
 - **Hibernate**: XML mapping for model class.
 - **Web Application**: using web.xml deployment descriptor file.

Hibernate and Spring Integration



We can simply integrate **hibernate application with spring application**.

In hibernate framework, we provide all the database information hibernate.cfg.xml file.

But if we are going to integrate the hibernate application with spring, we don't need to create the hibernate.cfg.xml file.

We can provide all the information in the applicationContext.xml file.

Advantages of Spring framework with hibernate

- The Spring framework provides HibernateTemplate class, so you don't need to follow so many steps like:
 - Create Configuration,
 - BuildSessionFactory,
 - Session,
 - beginning and committing transaction etc.
- So it saves a lot of code.
- **Solution by using HibernateTemplate class of Spring Framework:**

```
Employee e1=new Employee(111,"John",40000);  
hibernateTemplate.save(e1);
```

Methods of HibernateTemplate class

Method	Description
void persist(Object entity)	persists the given object.
Serializable save(Object entity)	persists the given object and returns id.
void saveOrUpdate(Object entity)	persists or updates the given object. If id is found, it updates the record otherwise saves the record.
void update(Object entity)	updates the given object.
void delete(Object entity)	deletes the given object on the basis of id.
Object get(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
Object load(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
List loadAll(Class entityClass)	returns the all the persistent objects.

Spring support for Hibernate

- Basically, in order to support Hibernate integration, Spring provides two key beans available in the **org.springframework.orm.hibernate5 package**
- **LocalSessionFactoryBean:**
 - Creates a Hibernate's **SessionFactory** which is injected into Hibernate-based DAO classes.
- **HibernateTransactionManager:**
 - Provides transaction support code for a **SessionFactory**.
- Programmers can use **@Transactional** annotation in DAO methods to avoid writing boiler-plate transaction code explicitly.

Spring JpaTemplate

Spring Data JPA API provides JpaTemplate class to integrate spring application with JPA.

JPA (Java Persistent API) is the sun specification for persisting objects in the enterprise application.

It is currently used as the replacement for complex entity beans.

It is basically used to integrate Hibernate with Spring

JPA Spring Annotations

- **@Entity** -Annotate all your entity beans with @Entity.

@Entity

```
public class Company {  
...  
}
```

- **@Service** - Annotate all your service classes with @Service.
 - All your business logic should be in Service classes.

@Service

```
public class CompanyServiceImpl implements CompanyService {  
...  
}
```

JPA Spring Annotations

- **@Repository**

- Annotate all your DAO classes with @Repository.
- All your database access logic should be in DAO classes.

@Repository

```
public class CompanyDAOImpl implements CompanyDAO {  
    ...  
}
```

- **@Transactional**

- Configure your transactions with @Transactional spring annotation.

@Transactional

```
public Company findByName(String name) {  
  
    Company company = companyDAO.findByName(name);  
    return company;  
}
```

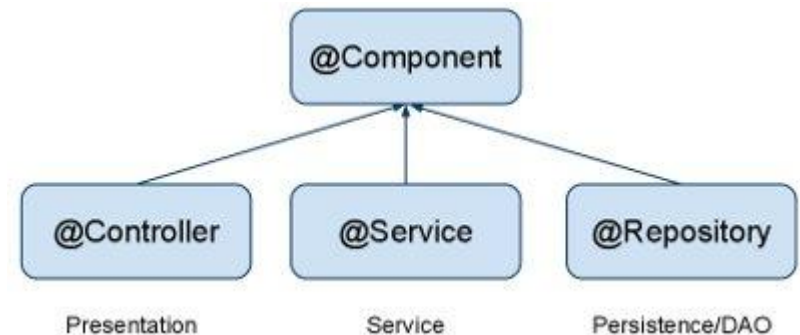
JPA Spring Annotations

- **@Component**

- Annotate your other components (for example REST resource classes) with @Component.
- @Component is a generic stereotype for any Spring-managed component.
- @Repository, @Service, and @Controller are specializations of @Component for more specific use cases,
- for example, in the persistence, service, and presentation layers, respectively.

@Component

```
public class ContactResource {  
    ...  
}
```



JPA Spring Annotations

- **@Autowired**
 - Let Spring auto-wire other beans into your classes using @Autowired annotation.
 - @Autowired by default is a type driven injection.

@Autowired

```
private CompanyDAO companyDAO;
```

Summary

- In this module, we have learnt:
 - Understanding Spring interaction with database using
 - JDBC
 - ORM (Hibernate)

References

- <http://www.theserverside.com/tt/articles/article.tss?l=SpringFramework>
- <http://static.springframework.org/spring/docs/>
- <http://www.javabeat.net/articles/71-introduction-to-spring-web-framework-3.html>
- <http://www.java2s.com/Code/Java/Hibernate/HibernateSpringHibernateTemplateFind.htm>

Thank You

